```
$aliceKey = sodium_crypto_box_keypair_from_secretkey_and_publickey(
    $privAlice,
    $pubBob
);
$bobKey = sodium_crypto_box_keypair_from_secretkey_and_publickey(
    $privBob,
    $pubAlice
);
```

Alice uses her private key and Bob's public key to create her encryption key pair. Bob does the same with his private key and Alice's public. At no point, do they need to share secrets: public keys are freely distributed; messages encrypted with a public key can only be decrypted by the matching private key (which is, of course, kept secret and never needs to be shared). Now we've set up our keys, we can use them to encrypt:

```
$message = 'The lamb lies down on Broadway.';
$nonce = random_bytes(SODIUM_CRYPTO_BOX_NONCEBYTES);
$encrypted_message = sodium_crypto_box($message, $nonce, $aliceKey);

print bin2hex($message) . PHP_EOL;
print bin2hex($encrypted_message) . PHP_EOL;

$decrypted_message = sodium_crypto_box_open(
    $encrypted_message,
    $nonce,
    $bobKey
);
print $decrypted_message;

Output:
546865206c...
0fe44c0f70...
The lamb lies down on Broadway.
```

And there we have it. `sodium_crypto_box()` scrambles our secret message with one key pair (and a random nonce value), `sodium_crypto_box_open()` unscrambles with the corresponding key pair, and the same single-use value.